

XSR 0.10 / External vectors

1. SVG is an interchange format

An external glyph is an explicit vector object. XSR imports its paths into its own geometry and draws them with PGF. No raster image, embedded PDF, font encoding or private-use character is created.

Three synthetic designs



The square, curved form and diamond above are tiny original SVG fixtures. They demonstrate multiple contours, quadratic curves and affine transforms.

Design space, not ink bounds



Both assets have a 200 by 200 design space and the same 1em advance. The sparse mark stays small inside its cell; it is not enlarged to fill the box.

Inline baseline

Before  after. Before  after.

The bottom of the default square is on the text baseline. The box has height 1em and depth zero. A rectangular viewBox preserves its aspect ratio, using geometric-mean normalization: $\sqrt{wh} = 1\text{em}$.

Direct API: `\xsrVectorGlyph{path/to/glyph.svg}`. Paths resolve from the XeLaTeX working directory. Source content joins the cache identity.

XSR 0.10 / External vectors

2. An optional external provider

The KAGE-facing commands below use the test suite's synthetic provider. They do not run KAGE and are not examples of KAGE stroke rendering. The same protocol can connect an independently installed KAGE-compatible wrapper.

Default serif option



Sans option



The synthetic producer changes a bar thickness to make style transport visible. An actual external producer decides how to interpret its style option.

Repeated request



One unique glyph/style/configuration is generated once and reused. The provider writes SVG; XSR validates it and caches the normalized vector response.

Protocol

```
provider --glyph ID --style STYLE --output PATH
```

The executable and prefix arguments come from a local JSON configuration. Python invokes an argument array with shell processing disabled.

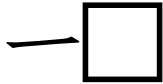
This PDF was prepared before compilation, then built with `xelatex -no-shell-escape`. Final compilation invokes no provider.

XSR 0.10 / External vectors

3. Missing glyphs remain visible

The default missing-glyph policy is **box**. XSR draws a hollow vector rectangle in the backend's nominal cell and logs a typed warning. The selected font's missing-glyph symbol is never used.

Khitan: Noto v1.000 lacks U+18CFF



The second cell is synthetic. The original two-column stack stays intact. Filler U+16FE4, SPACE and ZWSP remain controls or separators, never boxes.

Egyptian: Noto lacks visible sign U+1342F



The normal sign and missing sign still participate in horizontal composition. Egyptian controls and variation selectors do not request glyph cells.

External provider: no requested glyph



This fixture returns the documented missing-glyph status. Unavailable providers, provider failures and invalid SVG have separate diagnostic codes.

Strict mode

`\xsrMissingGlyphPolicy{error}` makes missing glyphs fatal. The normal default can be restored with **box**. Malformed direct SVG always raises a typed error; it is not silently replaced.

The synthetic geometry does not guess an absent historical shape. No KAGE implementation, GlyphWiki dataset or external font is bundled with XSR.